

Guião de Apoio 8

Comunicação Segura com SSL/TLS

Faculdade de Ciências da Universidade de Lisboa (FCUL)

Aplicações Distribuídas 2025/2026

1 Introdução

Uma das formas mais simples de garantir a comunicação segura entre aplicações distribuídas é usar o protocolo TLS (Transport Layer Security) que é uma versão mais moderna do SSL (Secure Sockets Layer)/ [2, 1]. O TLS funciona sobre TCP (Transmission Control Protocol) e faz uso de técnicas de criptografia para proteger a autenticação, a integridade e a confidencialidade dos dados transmitidos. A Figura 1 ilustra o protocolo.

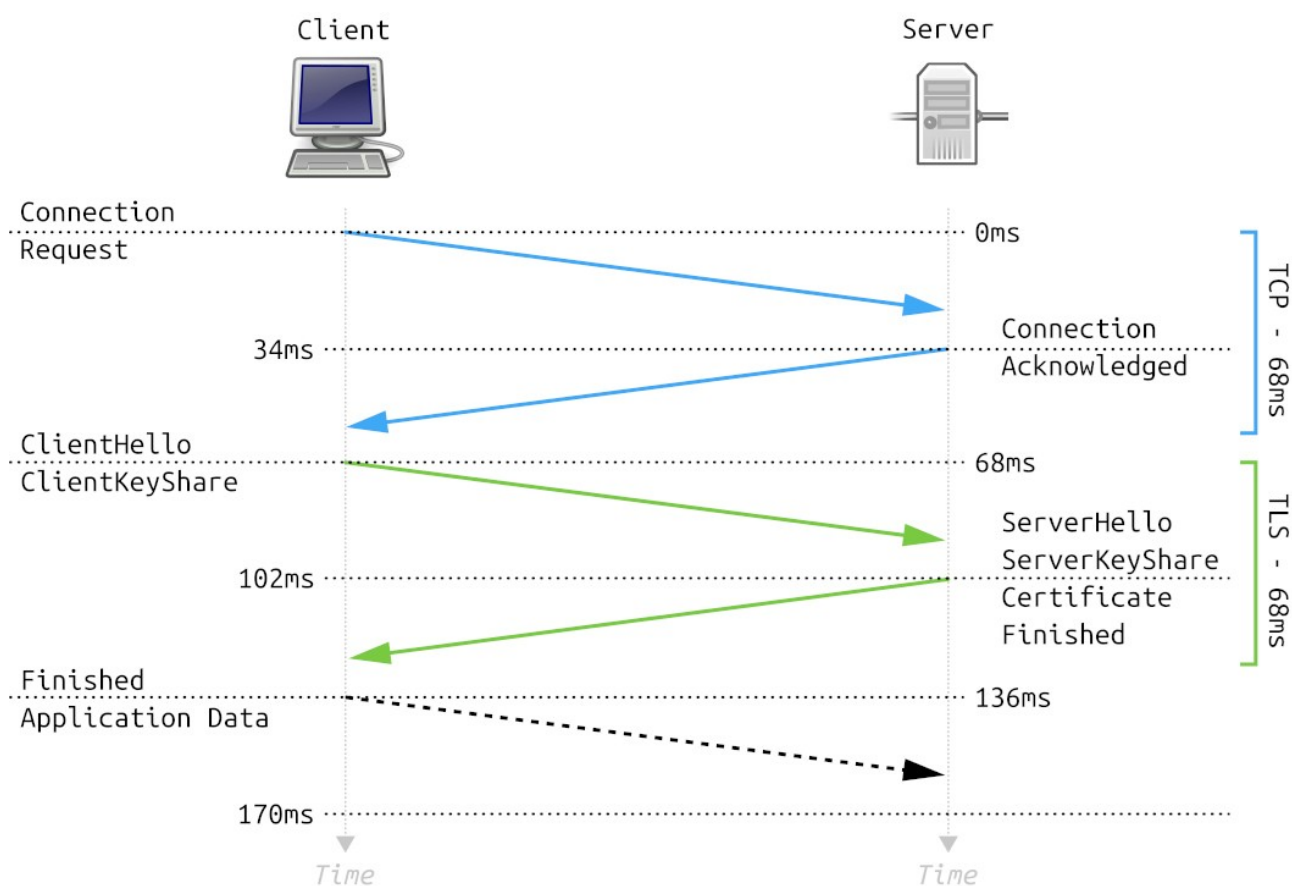


Figure 1: Protocolo TLS.

Não vamos discutir os detalhes do TSL neste documento, mas recomendamos os alunos a reverem os conceitos relacionados com o protocolo nos slides da última aula TP. Neste guião, vamos nos concentrar na implementação de uma comunicação segura por TCP.

2 criação de Chaves e Certificados Autoassinados

Vamos começar por criar uma CA (Certifica Authority) local para assinar certicados trocados entre cliente e servidor.

1. O primeiro passo é criar a chave privada da CA. É com ela que vamos "assinar" os certificados. Use o terminal.

```
openssl genrsa -out root.key 2048
```

Este comando gera a chave privada da CA, guardada no ficheiro `root.key`. Esta chave será utilizada posteriormente para assinar os certificados do servidor e do cliente.

2. Crie então um certificado auto-assinado para a CA:

```
openssl req -x509 -new -nodes -key root.key -sha256 -days 365 -out root.pem
```

Este comando cria o certificado da CA, guardado no ficheiro `root.pem`. Este certificado será usado para assinar e validar os certificados do servidor e do cliente.

3. De seguida, procede-se à geração das chaves privadas do servidor (`serv.key`) e do cliente (`cli.key`), respetivamente, utilizando os seguintes comandos:

```
openssl genrsa -out serv.key 2048
```

```
openssl genrsa -out cli.key 2048
```

4. Para que a CA possa assinar os certificados do servidor e do cliente, é necessário gerar, para cada um, um pedido de assinatura de certificado (CSR), utilizando os seguintes comandos. Em ambos os casos, deve ser indicado `localhost` no campo *Common Name*.

```
openssl req -new -nodes -key serv.key -sha256 -days 365 -out serv.csr
```

```
openssl req -new -nodes -key cli.key -sha256 -days 365 -out cli.csr
```

Um CSR (*Certificate Signing Request*) é um pedido que contém a chave pública e a informação de identificação de uma entidade. Este pedido é para ser enviado para a CA, que o assina e gera um certificado digital, permitindo a autenticação da entidade em comunicações seguras.

5. Na posse dos pedidos de assinatura de certificado do servidor (`serv.csr`) e do cliente (`cli.csr`), a CA pode proceder à geração dos certificados assinados. Este processo pode ser realizado para ambas as entidades através dos seguintes comandos:

```
openssl x509 -req -in serv.csr -CA root.pem -CAkey root.key -CAcreateserial -out serv.crt -days 365 -sha256
```

```
openssl x509 -req -in cli.csr -CA root.pem -CAkey root.key -CAcreateserial -out cli.crt -days 365 -sha256
```

Através deste processo, tanto o cliente como o servidor passam a dispor de um par de ficheiros composto por uma chave privada (`.key`) e um certificado digital (`.crt`).

Durante o estabelecimento de uma ligação segura, o cliente e o servidor trocam os seus certificados. Cada entidade verifica então se o certificado recebido foi assinado por uma CA de confiança. Para tal, utilizam o certificado da CA (`root.pem`), que contém a chave pública da mesma.

Se a assinatura for válida, a identidade do interlocutor é considerada confiável, permitindo assim a autenticação mútua. Após esta verificação, é estabelecida uma comunicação cifrada através do protocolo TLS.

3 Uso de SSL/TLS no Python

Para transformar um socket TCP num SSL/TLS basta criar um contexto `SSLContext` do módulo `ssl` do Python [3] e utilizar a função `wrap_socket`. O contexto recebe um conjunto de informações relativas ao TLS (e.g., versão do protocolo, caminhos para ficheiros com chaves e certificados) e a função `wrap_socket` recebe como parâmetro o socket TCP previamente criado e já ligado. A seguir, apresentamos trechos de códigos para concretizar clientes e servidores num cenário de autenticação unilateral. Neste caso, apenas o cliente verifica a identidade do servidor que comunica com qualquer cliente.

```
# ----- Codigo cliente -----#
import ssl

...
context = ssl.SSLContext(protocol=ssl.PROTOCOL_TLS_CLIENT)
context.verify_mode = ssl.CERT_REQUIRED
context.check_hostname = True
context.load_verify_locations(cafile='root.pem')
sslsock = context.wrap_socket(sock, server_hostname=HOST)
```

```
# ----- Codigo servidor -----#
import ssl

...
context = ssl.SSLContext(protocol=ssl.PROTOCOL_TLS_SERVER)
context.verify_mode = ssl.CERT_NONE
context.load_cert_chain(certfile='serv.crt', keyfile='serv.key')
sslsock = context.wrap_socket(conn_sock, server_side=True)
```

Note que o cliente deve ter acesso ao certificado da CA para verificar o certificado que o servidor envia (uma vez que este é assinado pela CA), enquanto o servidor deve ter acesso a sua chave privada (para provar a sua identidade) e ao seu certificado (para ser enviado aos clientes).

Os próximos dois trechos de código apresentam um código similar aos anteriores, mas agora para autenticação mútua entre cliente e servidor.

```
# ----- Codigo cliente -----#
import ssl

...
context = ssl.SSLContext(protocol=ssl.PROTOCOL_TLS_CLIENT)
context.verify_mode = ssl.CERT_REQUIRED
context.check_hostname = True
context.load_verify_locations(cafile='root.pem')
context.load_cert_chain(certfile='cli.crt', keyfile='cli.key')

sslsock = context.wrap_socket(sock, server_hostname=HOST)
...
```

```
# ----- Codigo servidor -----#
import ssl

...
context = ssl.SSLContext(protocol=ssl.PROTOCOL_TLS_SERVER)
context.verify_mode = ssl.CERT_REQUIRED
context.load_verify_locations(cafile='root.pem')
context.load_cert_chain(certfile='serv.crt', keyfile='serv.key')

sslsock = context.wrap_socket(conn_sock, server_side=True)
```

4 Exercícios

- Siga os passos da Secção 2 para criar todas as chaves e certificados necessários para utilizar comunicação segura entre servidores e clientes em Python.
- Copie os programas cliente e servidor apresentados a seguir e execute-os no computador.

```
# ----- Codigo cliente -----#
import sys, socket as s

if len(sys.argv) > 1:
    HOST = sys.argv[1]
    PORT = int(sys.argv[2])
else:
    HOST = 'localhost'
    PORT = 9999

while True:
    msg = input('Mensagem: ');
    if msg == 'EXIT':
        exit()

    sock = s.socket(s.AF_INET, s.SOCK_STREAM)
    sock.connect((HOST, PORT))

    sock.sendall(msg.encode())
    resposta = sock.recv(1024)
    print ('Recebi: %s' % resposta.decode())
    sock.close()
```

```

# ----- Codigo servidor -----#
import sys, socket as s

HOST = 'localhost'
if len(sys.argv) > 1:
    PORT = int(sys.argv[1])
else:
    PORT = 9999
sock = s.socket(s.AF_INET, s.SOCK_STREAM)
sock.setsockopt(s.SOL_SOCKET, s.SO_REUSEADDR, 1)

sock.bind((HOST, PORT))
sock.listen(1)

list = []

while True:
    (conn_sock, addr) = sock.accept()
    try:
        tmp = conn_sock.recv(1024)
        msg = tmp.decode()
        resp = 'Ack'

        if msg == 'LIST':
            resp = str(list)
        elif msg == 'CLEAR':
            list = []
            resp = Lista apagada
        else:
            list.append(msg)

        conn_sock.sendall(resp.encode())

        print ('list= %s' % list)
        conn_sock.close()
    except:
        print ('socket fechado!')
        conn_sock.close()

sock.close()

```

- (c) Modifique os programas anteriores para usarem SSL para comunicação, de tal forma que o cliente verifique a identidade do servidor (autenticação unilateral).
- (d) Modifique os programas anteriores para usarem SSL para comunicação com autenticação mútua.

Referências

References

- [1] OpenSSL Project. *OpenSSL Command Line Utilities Documentation*. Acedido em 2026. 2024. URL: <https://www.openssl.org/docs/apps/openssl.html>.
- [2] OpenSSL Project. *OpenSSL: Cryptography and SSL/TLS Toolkit*. Acedido em 2026. 2024. URL: <https://www.openssl.org/>.
- [3] Python Software Foundation. *ssl — TLS/SSL wrapper for socket objects*. Acedido em 2026. 2024. URL: <https://docs.python.org/3/library/ssl.html>.